

Bases de données avancées

Procédures, Fonctions et Packages



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction



Généralités -1

- Un bloc anonyme PL/SQL est un bloc «DECLARE –BEGIN –END » comme dans les exemples vus dans les chapitres précédents.
- Dans SQL*PLUS on peut exécuter directement un bloc PL/SQL anonyme en tapant sa définition
- Le plus souvent, on souhaite créer plutôt une **procédure ou une fonction nommée** pour réutiliser le code
- Une **procédure** est un ensemble de code PL/SQL nommé, défini par l'utilisateur et généralement stocké dans la BDD.
- Une **fonction** est identique à une procédure à la différence qu'elle retourne une valeur.
- Un **paquetage** est le regroupement (agrégat) de plusieurs procédures et fonctions dans un objet distinct.

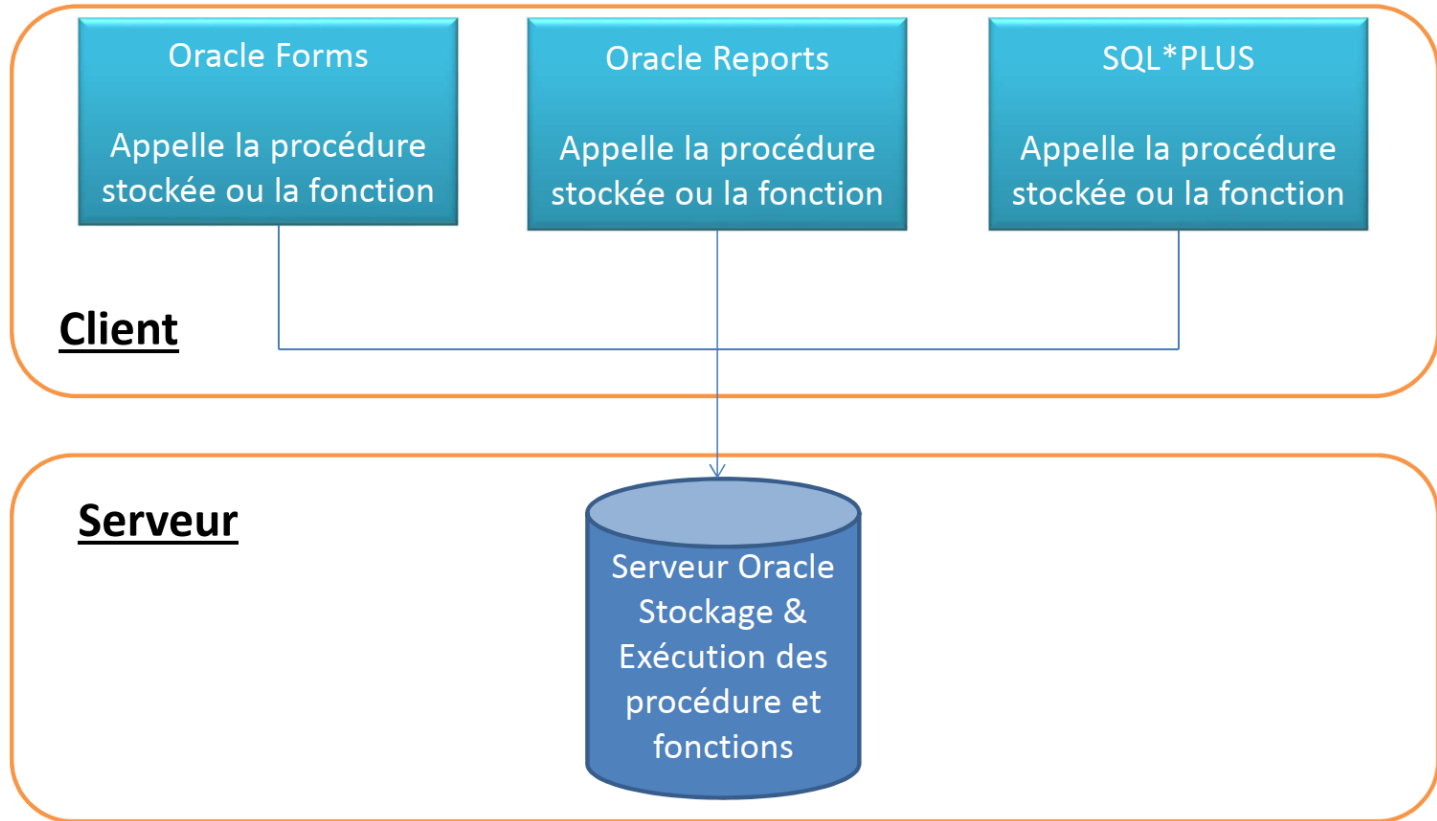


Généralités -2

- Une procédure stockée ou une fonction est un bloc stocké dans la base de données.
- Ce bloc peut être appelé par n'importe quel autre bloc PL/SQL que ce soit anonyme ou stocké aussi.
- On peut appeler les procédures et les fonctions par « sous programmes ».
- Les packages, procédures, ou fonctions peuvent être appelés depuis toutes les applications qui possèdent une interface avec ORACLE (SQL*PLUS, Pro*C, SQL*Forms, ...)



Généralités -3





Avantages procédures/fonctions

Les procédures (fonctions) permettent de :

- Réduire le trafic sur le réseau (les procédures sont locales sur le serveur)
- Mettre en œuvre une architecture client/serveur de procédures et rendre indépendant le code client de celui des procédures (à l'API près)
- Masquer la complexité du code SQL (simple appel de procédure avec passage d'arguments)
- Sécuriser l'accès aux données (accès à certaines tables seulement à travers les procédures)
- Optimiser le code (les procédures sont compilées avant l'exécution du programme et elles sont exécutées immédiatement si elles se trouvent dans la SGA (zone mémoire gérée par ORACLE).
- De plus une procédure peut être exécutée par plusieurs utilisateurs.



Avantages packages

Les packages permettent de :

- Regrouper des procédures ou des fonctions (ou les deux). On évite ainsi d'avoir autant de sources que de procédures.
- De travailler en équipes et l'architecture applicative peuvent donc plus facilement s'organiser du côté serveur, où les packages regrouperont des procédures à forte cohésion intra (Sélection de tous les articles, Sélection d'un article, Mise à jour d'un article, Suppression d'un article, Ajout d'un article).
- Les packages sont utilisés comme de simples librairies par les programmes clients (librairies distantes « sur le serveur »)

2

Procédures



Syntaxe de création

```
CREATE [OR REPLACE] PROCEDURE <nom_procedure>  
    [(argument [IN|OUT|IN OUT] type [DEFAULT valeur]  
    ...  
    argument [IN|OUT|IN OUT] type)]  
IS|AS  
    <partie_déclaration>  
BEGIN  
    <corps_procedure>  
[EXCEPTION  
    <gestion d'exception>]  
END;
```



Paramètres d'une procédure

- **CREATE** indique que l'on veut créer une procédure stockée dans la base
- La clause facultative **OR REPLACE** permet d'écraser une procédure existante portant le même nom
- **nom procédure** est le nom donné par l'utilisateur à la procédure
- **AUTHID** détermine si la procédure s'exécute avec les privilèges de son propriétaire (DEFINER, par défaut) ou de l'utilisateur courant (CURRENT_USER)



Liste d'arguments

- Ce sont les arguments ou paramètres de la procédure. On peut en déclarer autant qu'on le souhaite.
- On déclare un argument suivi de son **mode** qui peut être:
 - **IN**: pour les paramètres entrés seulement. Ne sera pas retourné dans le résultat. Il indique que le paramètre transmis par le programme appelant n'est pas modifiable par la procédure
 - **OUT**: pour les paramètres de sortie seulement. Il indique que le paramètre est modifiable par la procédure
 - **IN OUT**: les deux; les paramètres peuvent être d'entrée et retournés en résultat.
- **datatype** représente le type SQL ou PL/SQL du paramètre. On fait suivre le mode par le type de l'argument (number, date, etc.).
- **nom paramètre** est le nom donné par l'utilisateur au paramètre transmis



Exemple

create or replace procedure listTypeJob(vjob IN varchar2, result OUT number) IS

BEGIN

SELECT count(*) INTO result FROM emp where job=vjob;

END;

/



```
SQL>
SQL> create or replace procedure listTypeJob(vjob IN varchar2, result OUT number) IS
2  BEGIN
3      SELECT count(*) INTO result FROM emp where job=vjob;
4  END;
5  /
6
7  Procédure créée.
```



Appel sous SQL*PLUS

```
SQL> VARIABLE RESULT NUMBER; Déclarer la variable
SQL>
SQL> EXECUTE listTypeJob('MANAGER',:result); Exécuter la procédure
Procédure PL/SQL terminée avec succès.

SQL>
SQL> PRINT result; Visualiser le résultat

      RESULT
-----
          3

SQL>
```



Appel dans un bloc PL/SQL

```
SET SERVEROUT ON
DECLARE
    result NUMBER;
    vjob    emp.job%TYPE;
BEGIN
    vjob:='MANAGER';
    listTypeJob(vjob,result);
    DBMS_OUTPUT.PUT_LINE('nbre : ' || result);
END;
/
```



Description des paramètres

```
SQL>
SQL> desc listTypeJob;
PROCEDURE listTypeJob
  Nom d'argument          Type          E/S par défaut ?
-----
UJOB                      VARCHAR2      IN
RESULT                    NUMBER        OUT
SQL>
```



Exercice

1. Ecrire une procédure avec en paramètre d'entrée le numéro de département et qui permet d'afficher la liste des employés de ce département. N'oubliez pas de gérer les exceptions (département n'existe pas...)
2. Ecrire une procédure permettant de supprimer un département passé en argument. N'oubliez pas de gérer les exceptions (département n'existe pas ,...)

3

Fonctions



Création d'une fonction

- La création d'une fonction est quasiment identique à celle d'une procédure stockée, avec une simple différence c'est qu'une fonction doit avoir une valeur de retour.
- Donc on l'affecte à une variable, par exemple: **Nom_cours := GetNom(p_ccode);**
- La fonction GetNom prend un seul paramètre (le code du cours) et retourne en résultat le nom du cours correspondant.
- Une fonction peut être utilisée n'importe où avec une expression: affectation, IF, traitement itératif, etc.
- elle a la même syntaxe qu'une procédure mais on ajoute un **return**



Syntaxe de création

```
CREATE [OR REPLACE] Function <nom_fonction>
    [(argument [IN|OUT|IN OUT] type [DEFAULT valeur]
    ...
    argument [IN|OUT|IN OUT] type)]
    RETURN <type_valeur_retournée>
IS|AS
    <partie_déclaration>
BEGIN
    <corps_fonction>
[EXCEPTION
    <gestion d'exception>]
Return <valeur>;
END;
```



Exemple

```
create or replace function AugSal(vempno emp.empno%type,vporcent
NUMBER) return number
IS
    vsal emp.sal%type;
BEGIN
    SELECT sal INTO vSal FROM emp WHERE empno = vempno ;
    vsal :=vsal + vsal*vporcent;

    return vSal;
END;
```



Remarques

- L'appel à la fonction et son exécution se passe exactement de la même manière qu'une procédure.
- Pour exécuter un sous programme (fonction ou procédure) d'un autre schéma il faut :
 - détenir le privilège EXECUTE sur le sous programme en question :
 - exemple : **GRANT EXECUTE ON nomPrcedure to hr ;**
 - ou **EXECUTE ANY PROCEDURE to hr;**
 - mentionner le schéma contenant le sous programme à l'appel de ce dernier :
 - exemple : **scott.AugSal(7900,50);**



Exercice

- ◎ Ecrire une fonction qui affiche la moyenne des salaires des employés d'un département dont le numéro est transmis en paramètre.



Corrigé

```
create or replace function avgSal(vdeptn number) return number is
vsal emp.sal%type;
BEGIN
    select avg(sal) into vsal from emp where deptno=vdeptn ;
    -- si deptno n'existe pas
    if SQL%notfound then
        raise_application_error(-20001, ' le num de dept' || vdeptn );
    end if ;
    return vsal;
END;
/
-- Appel de la fonction :
declare vsal number;
begin
    vsal:=avgSal(20);
    dbms_output.put_line(vsal);
end;
/
```



Opérations sur les fonctions/procédures



Suppression de procédures ou fonctions :



DROP PROCEDURE nom_procedure ou **DROP FUNCTION** nom_fonction



pour consulter le code d'une fonction ou procédure



consulter USER_SOURCE :



SELECT text FROM USER_SOURCE where name='nom_fct ou procédure';



Pour avoir la liste des fonctions ou procédure d'un schéma :



Interroger USER_OBJECTS :



select object_name from user_objects where object_type='FUNCTION';



select object_name from user_objects where object_type='PROCEDURE';

4

Packages



Généralités -1

- Un paquetage (package) est un composant qui regroupe plusieurs objets (variables, exceptions, curseurs, fonctions, procédures ...)
- Un paquetage est organisé en deux parties distinctes:

Une partie spécification

- qui permet de spécifier à la fois les fonctions et procédures publiques ainsi que les déclarations des types, variables, constantes, exceptions et curseurs utilisés dans le paquetage et visibles par le programme appelant .

Une partie corps

- qui contient les blocs et les spécifications de tous les objets publics listés dans la partie spécification
- Cette partie peut inclure des objets qui ne sont pas listés dans la partie spécification, et sont donc privés.
- Cette partie peut également contenir du code qui sera exécuté à chaque invocation du paquetage par l'utilisateur



Package = Encapsulation

SPECIFICATIONS

- Partie publique d'une package
- Déclarations des objets
- Accès avec le droit EXECUTE

BODY

- Partie privée d'un package
- Définition des objets
- Peut contenir une partie initialisation des objets



Syntaxe d'un package

```
CREATE OR REPLACE PACKAGE nom_package IS  
    définitions des types utilisés dans le package;  
    prototypes de toutes les procédures et fonctions du package;  
END nom_package;  
/
```

```
CREATE OR REPLACE PACKAGE BODY nom_package IS  
    déclaration de variables globales;  
    définition de la première procédure;  
    définition de la deuxième procédure;  
    etc. ...  
END nom_package;  
/
```



Explications

Un package est composé d'un en tête et d'un corps :

- L'en tête comporte les types de données définis et les prototypes de toutes les procédures (fonctions) du package.
- Le corps correspond à l'implémentation (définition) des procédures (fonctions).
- Le premier END marque la fin de l'en tête du package. Cette partie doit se terminer par / pour que l'en tête soit compilé.
- Le corps (body) du package consiste à implémenter (définir) l'ensemble des procédures ou fonctions qui le constitue. Chaque procédure est définie normalement avec ses clauses BEGIN ... END.
- Le package se termine par / sur la dernière ligne.



Exemple -1

CREATE OR REPLACE PACKAGE scott.gestEmp is

 FUNCTION AugSal(vempno emp.empno%type,vporcent NUMBER)
 return number;

 procedure listTypeJob(vjob IN varchar2, result OUT number);

END gestEmp ;

Spécification



Exemple -2

```
CREATE OR REPLACE PACKAGE BODY scott.gestEmp is

function AugSal(vempno emp.empno%type,vporcent NUMBER) return number IS
    vsal emp.sal%type;
    BEGIN
        SELECT sal INTO vSal FROM emp WHERE empno = vempno ;
        vsal :=vsal + vsal*vporcent;

        return vSal;
    END;

procedure listTypeJob(vjob IN varchar2, result OUT number) IS
    BEGIN
        SELECT count(*) INTO result FROM emp where   job=vjob;
    END;

End gestEmp ;
```

Corps du package



Appel procédures/fonctions



Préfixer la fonction ou la procédure par le nom du package :

```
SQL> begin :vsal := gestEmp.AugSal(7900,60); END;  
2 /
```

Procédure PL/SQL terminée avec succès.

```
SQL> print vsal;
```

```
      VSAL
```

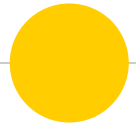
```
-----  
      57950
```




Droits sur un package

- Pour créer un paquetage dans son propre schéma, il faut détenir le privilège **CREATE PROCEDURE**
- Pour pouvoir créer un paquetage dans un autre schéma, il faut avoir le privilège **CREATE ANY PROCEDURE**
- Pour qu'un user puisse exécuter une fonction ou procédure d'un package :

GRANT EXECUTE on nomPackage.nomProcedure to user



Fin du chapitre 5